



THE UNITED REPUBLIC OF TANZANIA



PRESIDENT'S OFFICE–PUBLIC SERVICE MANAGEMENT AND GOOD GOVERNANCE

e-Government Authority (e-GA)

Research, Innovation and Development Center (RIDC)

SYSTEM DESIGN DOCUMENT (SDD)

October 2026

PROJECT: A BLOCKCHAIN-BASED AI REPUTATION ENGINE FOR DECENTRALIZED
CREDIT ASSESSMENT

TABLE OF CONTENTS

1. INTRODUCTION	4
1.1 Introduction.....	4
1.2 Project Overview.....	4
1.3 Purpose.....	4
1.4 Scope.....	4
1.5 Audience.....	5
1.6 References.....	5
2. SYSTEM ARCHITECTURE.....	6
2.1 Architectural Overview.....	6
2.2 Design Principles.....	6
2.4 Hardware Design.....	9
2.5 Software Design.....	10
2.5.1 Lender Application Process.....	10
2.5.2 Software Interface Flow Chart.....	10
2.6 Security Software Architecture.....	11
2.7 System Integrity Controls.....	12
2.8 Performance Software Architecture.....	12
2.9 Technology Stack.....	13
3. DATA FLOW DESIGN	14
3.1. Data Flow Design in DAIRE.....	14
3.3 Data Flow Benefits.....	15
4. UNIFIED MODELING LANGUAGE DESIGN	16
4.1 Entity-relationship diagram.....	16
4.1.1 Data Dictionary.....	18
4.2 User Case Diagram.....	23
4.2.1 Actors.....	23
4.2.2 Use Case Descriptions.....	24
5. SYSTEM INTERFACE DESIGN.....	25
5.1 User Interface Design.....	25
5.1.1 Customer Portal (Lender).....	25
5.1.2 Lender Administration Portal.....	26
5.1.3 Central Operations Console.....	26

5.2 External Interfaces.....	26
5.2.1 NIDA Verification API.....	26
5.2.2 Central Lender Data and Broadcast APIs.....	26
5.2.3 Lender Backend API Catalogue.....	27
5.2.4 Central System API Catalogue.....	28
6.SYSTEM COMPONENTS.....	28
6.1 Frontend.....	28
6.2 Backend.....	29
6.3 Database.....	29
6.4 AI Reputation Engine.....	29
6.5 Blockchain Scoring Service.....	30
6.6 NIDA Registry System.....	31
6.7 Central System Modules.....	31
7. SECURITY DESIGN.....	31
7.1 Authentication and Authorization.....	31
7.2 Network and Interface Security.....	31
7.3 Consent Management.....	32
7.4 Data Privacy and Compliance.....	32
8. APPENDICES.....	32
8.1 Glossary of Terms.....	32
8.2 Compliance and Licensing.....	33
8.2.1. Software Licensing.....	33
8.3 Data Privacy and Compliance.....	34

1. INTRODUCTION

1.1 Introduction

This is the Software Design Document (SDD) for the Decentralized AI Reputation Engine (DAIRE). This document presents the general design of DAIRE, including the system overview, the purpose of the SDD, the scope of the SDD and its intended audience. The document aims to provide a clear and structured description of the architecture and components of the platform, ensuring that all stakeholders have a common understanding of the design before development, integration and deployment.

1.2 Project Overview

Right now, many entrepreneurs, small groups of women and youth, and small-scale farmers in Tanzania cannot obtain credit from formal lenders because they have no formal credit history. Each lender sees only the customer data it holds itself, so a person who repays well at one institution is still treated as unknown by another. Lenders also have no common, trusted way of confirming who the customer is and how the customer has behaved across the market.

DAIRE solves these problems by connecting the systems of the Financial Service Providers (FSPs) to one Central System. A customer is first verified against the national identity registry (NIDA). With the customer's consent, each lender then sends its own data (accounts, transactions, loans and repayments) to the Central System, which merges the records using a single borrower reference. An AI reputation model and a blockchain smart contract analyse the merged record and produce one credit score between 300 and 850. The identical result is then broadcast back to every lender that is linked to the borrower, so that all lenders make decisions from the same trusted evidence.

The platform currently links four lender systems (CRDB-01, MoFinance-004, NMB-01 and NMB-02) which use the same lender design but different user interfaces and lender identities, together with the NIDA registry system, the DAIRE Central System, the AI engine and the blockchain scoring service.

1.3 Purpose

This document describes the architecture and system design of DAIRE. It will serve as a reference throughout the design, development, testing, integration and implementation phases of the project. The primary goal of this document is to present the high-level and detailed design of the system, including the software architecture, system components, data flow, data model, interfaces and security considerations. This ensures that developers, testers and stakeholders have a shared understanding of the solution being built and that the final implementation aligns with the requirements defined in the DAIRE System Requirements Specification (SRS).

1.4 Scope

The system design document for DAIRE aims to describe the following:

1. **Functional Requirements:** Core features such as customer registration with NIDA verification, account and transaction management, loan applications, consent management, data exchange with the Central System, AI and blockchain credit scoring, and broadcast of results to lenders.

2. **Operating Environment:** Description of the hardware and software environments where the platform will run, including lender servers, the NIDA server, the Central server, the blockchain node and client web browsers.
3. **System and Subsystem Architecture:** A detailed breakdown of the lender subsystem, the NIDA subsystem and the Central subsystem, including the user interface layer, application logic, database layer and the integration with the AI engine and the blockchain gateway.
4. **Files and Database Design:** The structure and organization of customer, account, loan, repayment, consent and synchronization data in each lender database and in the NIDA registry.
5. **User Interfaces:** The design and flow of the main screens of the customer portal, the lender administration portal and the Central operations console.
6. **Security Design:** Authentication, authorization, network trust, consent enforcement and privacy controls.

1.5 Audience

This design documentation is intended for developers, testers, project managers and other key stakeholders involved in the project.

- ❖ Developers will use this document to understand the system design and architecture to implement and extend the solution according to the requirements.
- ❖ Testers will rely on this document to verify that all functional and non-functional requirements are met and to develop test cases that ensure system reliability and security.
- ❖ Project Managers and Stakeholders will use it to track system development progress, understand the project's scope and monitor timelines until completion.
- ❖ Financial Service Provider (FSP) IT Teams will use it to integrate their own lender system with the Central System using the interface contracts in Chapter 5.
- ❖ Support and Maintenance Teams will refer to it when troubleshooting issues or implementing future updates to the system.

1.6 References

1. DAIRE System Requirements Specification (SRS), Version 1.0, e-Government Authority RIDC, October 2026.
2. DAIRE Central System README and Lender Subsystem Integration Contract (LENDER_SUBSYSTEM_README).
3. DAIRE AI Data Contract (JSON in / JSON out), AI_DATA_CONTRACT.
4. DAIRE Architecture and Scoring Notes (ARCHITECTURE and STEP-02-SCORING).
5. DaireCreditScore smart contract, Solidity source of the credit scoring logic.
6. NIDA system source: registry models, serializers, views and permissions.

1.7 Design Assumptions and Constraints

Type	Description
Assumption	Every lender holds the customer's NIDA number and shares one borrower reference with Central so that records can be merged exactly.
Assumption	The NIDA registry is available on the trusted network and answers verification requests only for callers on its IP allow-list.
Assumption	Customers have given consent before any lender pushes data to the Central System.
Constraint	The credit score is calculated in one place only (the blockchain contract) so that all lenders receive the same value.
Constraint	Raw personal identity data (NIDA number, names, phone, address) is not sent to the AI engine or to the blockchain; only derived dimensions are sent.
Constraint	Services run on a closed network and use fixed IP addresses and ports for the demonstration deployment.

2. SYSTEM ARCHITECTURE

2.1 Architectural Overview

DAIRE is designed using an Agile development methodology, which allows iterative development, continuous feedback and fast delivery of features. The platform is made of independent systems that talk to each other through REST APIs and a blockchain RPC connection. Each lender system and the NIDA system is a Django (Python) application with Django REST Framework and a PostgreSQL database, and has its own Angular web interface. The Central System has a Django REST backend.

2.2 Design Principles

1. **Modularity:** The platform is divided into independent systems and apps such as accounts, customers, loans, consent and daire_integration. Each module can be developed, tested and maintained separately.
2. **Single Source of Truth:** Credit scoring is calculated once, by the smart contract, and the same result is delivered to all lenders linked to the borrower.
3. **Privacy by Design:** Data leaves a lender only with an active consent and only the allowed data categories are shared; the AI and the blockchain receive derived features, never raw identity data.
4. **Scalability:** A new lender is added by duplicating the lender subsystem and registering it in Central; a new AI engine or blockchain network can replace the current one through configuration.

5. Security: Hashed PINs, one-time passwords, JWT tokens, IP allow-lists and role-based access protect every system.
6. Auditability: Every data exchange, consent change and score is stored with a reference and time so that each decision can be traced.

2.3 Architectural Diagram

DAIRE CONCEPTUAL DIAGRAM

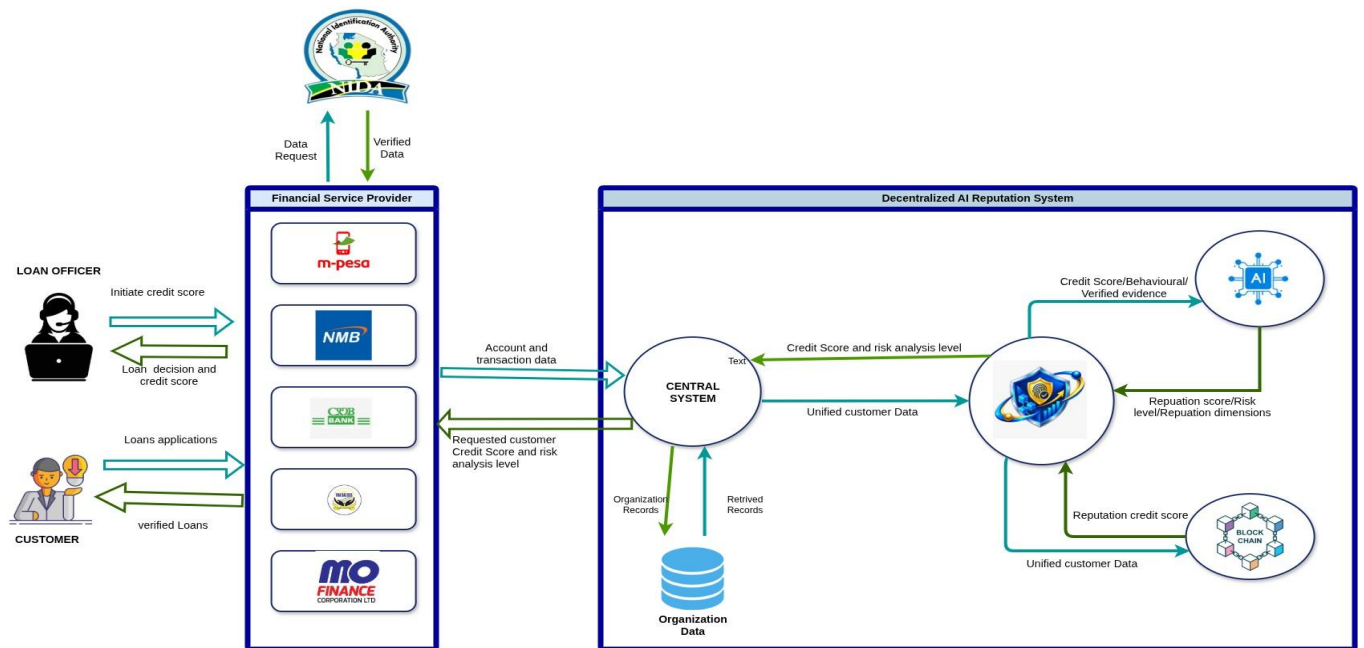


Figure 1: Architectural Diagram

1. Customer Opens the Lender Portal

- ❖ The process starts when a customer opens the lender portal (the Angular application of CRDB, NMB or MoFinance) in a browser.
- ❖ The portal is the main interface for registration, loans, accounts, consent and viewing the credit score.

2. Portal Sends the Request to the Lender Backend

- ❖ The portal sends the customer's request (registration, login, loan application, consent) to the lender backend (Django + DRF).

3. Lender Pushes Consented Data to Central

- ❖ When consent is active, the lender backend sends the borrower's accounts, loans and repayments to the Central System using one borrower reference.

4. Central Runs the Scoring Services

- ❖ Central merges the data, generates the credit features and sends them to the AI engine and to the blockchain scoring contract.

5. Result Returns to the Lender and the Customer

- ❖ Central broadcasts the same result to every linked lender and the portal shows the credit score, risk band and notifications to the customer.

6. Lender Backend Verifies Identity with NIDA

- ❖ During registration the lender backend calls the NIDA registry with the NIDA number and names.

7. NIDA Returns the Verification Result

- ❖ NIDA answers matched or not matched together with the citizen record (date of birth, gender, region, district, ward).

8. Customer Support through the Operator Console

- ❖ Operators use the Central operations console to monitor lenders, data exchanges and assessments, and to support lenders when a delivery fails.

9. Blockchain Node Executes the Smart Contract

- ❖ The Hardhat node executes the DaireCreditScore contract called through the Express middleware and stores the score, dimensions and risk band on the ledger.

2.4 Hardware Design

The hardware for DAIRE consists of the servers that host each system and the devices that customers, lender staff and Central operators use to access them. These devices must have stable network access to the servers. On the server side, each system requires reliable hosting with enough capacity to handle concurrent users, loan applications, data pushes and scoring requests.

Host	Role	Services Running
172.17.16.68	Central System server	Central Django backend, Angular operations console (port 4200), PostgreSQL (port 5434), Express middleware (port 5000), Hardhat node (port 8545)
172.17.16.70	CRDB and NIDA server	CRDB lender backend (port 8002) and Angular portal (port 4600); NIDA registry (port 8020) with its PostgreSQL database
172.17.16.76	NMB server	NMB lender backend (port 8002) and Angular portal (port 4300), serving the NMB-01 and NMB-02 lenders
172.17.16.76	MoFinance server	MoFinance lender backend and portal following the same lender design
172.17.16.76	Customers, lender staff, operators	Computers, tablets and phones with a modern web browser

2.5 Software Design

2.5.1 Lender Application Process

As shown in the architectural diagram, the DAIRE lender process starts when a customer opens the lender portal and registers or logs in with an account number, a four-digit PIN and a one-time password. Once logged in, the customer can perform several actions:

1. Register with NIDA: Provide the NIDA number and names, which are verified against the national registry before the account is created.
2. Manage Accounts: View balances and transactions, make deposits and withdrawals and transfer money.
3. Apply for Loans: Choose a loan type, amount, purpose and period and follow the application until it is approved or rejected.
4. View the Credit Score: See the DAIRE credit score, risk band, credit history and notifications.

2.5.2 Software Interface Flow Chart

In the logical design of the system, the flowchart maps the entire process flow of how customers and loan officers interact with DAIRE. The flowchart shows how users open the portal, log in or register, reach the dashboard, apply for a loan, grant consent, have the application decided by an officer, and have their data pushed to the Central System for AI and blockchain scoring before the score is displayed back in the portal.

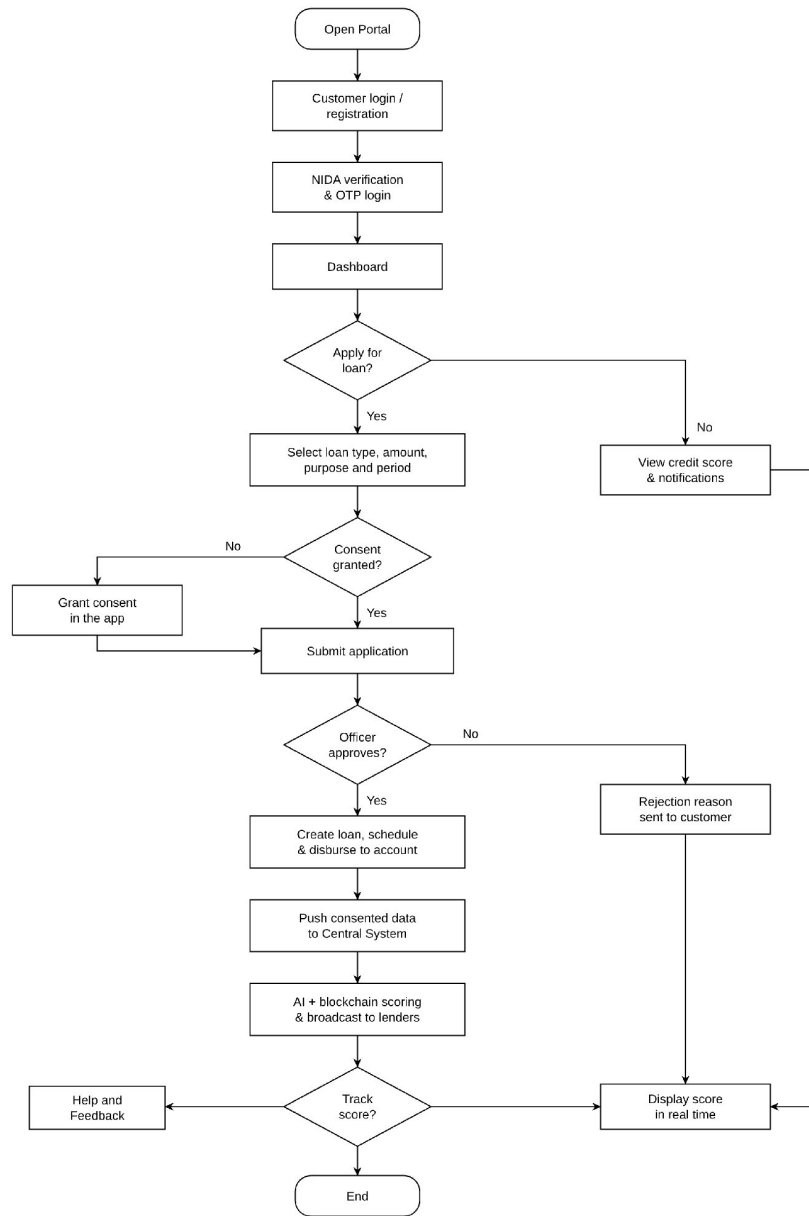


Figure 2: Flow Chart Diagram

2.6 Security Software Architecture

DAIRE applies several principles to ensure the security and privacy of customer data and transactions:

Requirement	Description
User Authentication and Authorization	Customers log in with an account number and a four-digit PIN followed by a one-time password (OTP). Administrators log in with their credentials.
Access Control	Access to features is restricted by role (customer, loan officer,

Requirement	Description
	administrator, Central operator). Customers can see only their own accounts, loans and consents. Figure 10: Sequence Diagram For Scoring and Broadcast Module
Encryption	PINs are stored only as salted hashes. Sensitive data is encrypted in transit (HTTPS/TLS) in production and secrets such as API keys and database passwords are kept in environment files, never in source code.
Consent Enforcement	A lender pushes data to Central only when the customer has an active consent. Revoking the consent stops all further pushes immediately.

2.7 System Integrity Controls

The following integrity controls help maintain the reliability and safety of DAIRE:

Requirement	Description
Secure and Standard Coding	Security considerations are integrated into every stage of development. The code uses the Django and Django REST Framework security features (input validation, ORM queries, permission classes) to minimize vulnerabilities.
Secure Credentials	PINs and passwords are stored with strong hashing and salting so that they cannot be recovered even if the database is compromised.
Data Integrity	Money movements use atomic database transactions. Borrower records are merged only on an exact borrower reference match and conflicts are logged instead of silently overwritten.
Immutable Score Record	The score, dimensions, risk band and timestamp are written to the blockchain ledger and cannot be changed afterwards, so every lender can verify the same value.
Strong Authentication Mechanisms	A one-time password valid for ten minutes is required for registration confirmation, login and password reset, adding an extra layer of security.

2.8 Performance Software Architecture

The following components are designed to optimize the performance of DAIRE, ensuring that customers, lenders and operators enjoy a smooth, reliable and responsive experience:

Requirement	Description
Scalable Infrastructure	Each lender system is deployed separately, so load is spread across lenders. A system can be scaled by adding servers or by

Requirement	Description
	moving its database to a stronger machine as the number of customers grows.
Load Balancing	Requests to a busy backend can be distributed across several application instances placed behind a reverse proxy to avoid overload and improve response time.
Session Management	JWT tokens (access token 12 hours, refresh token 7 days) keep users logged in without server-side session storage, which keeps response times fast.
Pagination	List endpoints return pages of results (20 items by default in Central and 50 in NIDA) so that large collections do not slow down the interfaces.
Asynchronous Delivery	Broadcasts to lenders are delivered one by one; a failure at one lender is logged and does not stop the delivery to the others.
Performance Monitoring	Server response times, API latency, database query efficiency and delivery status are monitored through the audit logs, data-exchange logs and the Central dashboard.
Caching	Frequently used reference data such as lender lists and routing policies can be cached temporarily to reduce repeated database queries.

2.9 Technology Stack

Layer	Technology	Used For
Lender and NIDA backend	Python, Django, Django REST Framework, simplejwt	Business logic, REST API, JWT authentication
Central backend	Python, Django, Django REST Framework	Borrower merge, assessments, data exchange, routing policies
Web interfaces	Angular (TypeScript)	Customer portal, lender administration portal, Central operations console
Database	PostgreSQL	Lender data, NIDA registry, Central data
AI engine	scikit-learn Random Forest (joblib model)	Reputation label and default probability from 11 features
Blockchain middleware	Node.js with Express	Bridge between Central and the smart contract
Smart contract	Solidity (DaireCreditScore) on Hardhat node	Dimension scoring, weighted score 300-850 and risk band

3. DATA FLOW DESIGN

3.1. Data Flow Design in DAIRE

The Data Flow Design describes how data moves through DAIRE, identifying the sources, destinations, processes and storage. The diagram illustrates the interaction between external entities (lenders, the NIDA registry, the AI engine, the blockchain and the operator), the Central System and the central database.

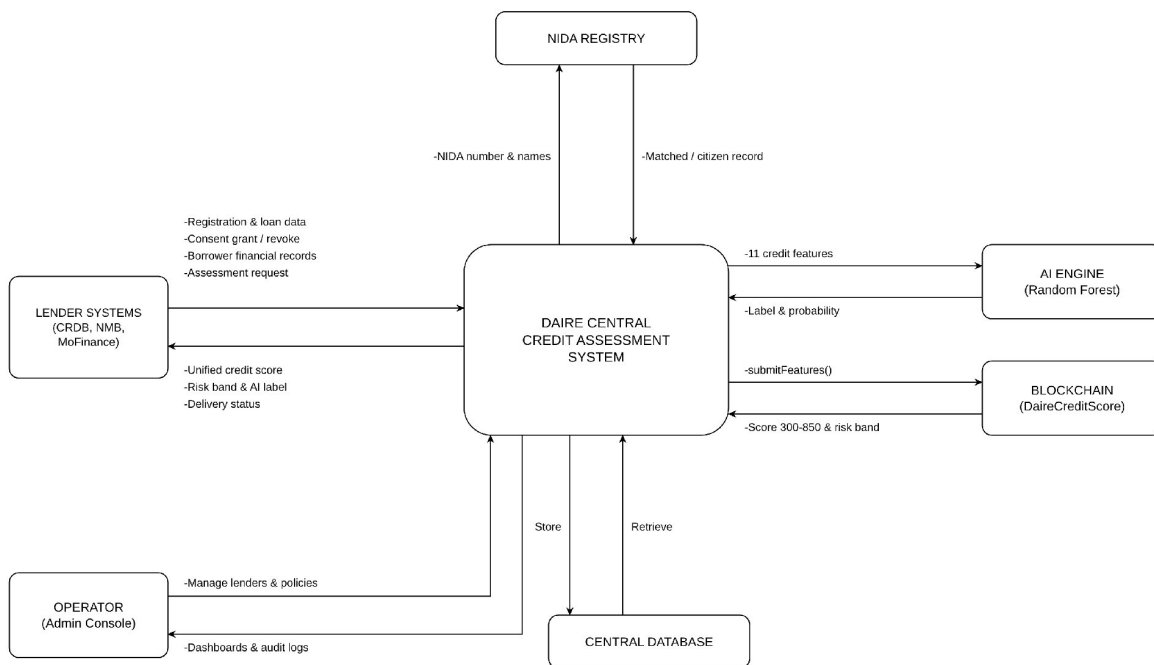


Figure 3: Data Flow Diagram

3.2 Data Flow Description

a. Registration and Identity Flow

1. **Input:** Customer details with the NIDA number and names.
2. **Process:** The lender backend validates the format, calls the NIDA registry, compares the names, creates the customer and primary account and stores a hashed PIN.
3. **Output:** Success message with the account number, or an error stating that the identity was not matched.

b. Loan Application Flow

1. **Input:** Loan type, amount, purpose and period.
2. **Process:** The lender checks the profile and consent, saves the application and notifies an officer who approves or rejects it; on approval the loan, schedule and disbursement are created.
3. **Output:** Decision with the reason, loan schedule and updated account balance.

c. Consent Flow

1. **Input:** Customer decision to grant, revoke or extend consent.
2. **Process:** The lender stores the consent with its version, dates and the customer's IP address and writes an audit record.
3. **Output:** Updated consent status; pushes to Central are enabled or stopped.

d. Data Exchange Flow

1. **Input:** Accounts, transactions, loans and repayments of a consenting customer.
2. **Process:** The lender builds the payload with the borrower reference and posts it to Central, which merges it with records from other lenders and stores a data-exchange log.
3. **Output:** Acknowledgement to the lender and an updated unified borrower record.

e. Scoring Flow

1. **Input:** Assessment request for a borrower reference.
2. **Process:** Central generates 11 features, sends them to the AI engine and to the blockchain contract and stores both results against the assessment.
3. **Output:** AI reputation, blockchain score, dimensions and risk band.

f. Broadcast Flow

1. **Input:** The assessment result and the list of lenders linked to the borrower.
2. **Process:** Central posts the identical result to each lender's receive endpoint and records the delivery status.
3. **Output:** Each lender shows the same score to its customer; delivery status per lender is logged.

3.3 Data Flow Benefits

1. **Clarity in system interaction:** Shows how external entities (lenders, NIDA, AI and blockchain) connect with the Central System.

2. **Traceability:** Ensures each data flow (input and output) has a clear path and a log entry, reducing ambiguity.
3. **Scalability:** The central database allows modular growth (more lenders, new data categories, other AI engines).
4. **Reliability:** Identical results are delivered to all lenders, and a failure at one lender does not stop the others.
5. **Privacy:** Only consented data categories leave a lender, and only derived features reach the AI and the blockchain.

4. UNIFIED MODELING LANGUAGE DESIGN

4.1 Entity-relationship diagram

The entity-relationship diagram on the next page shows the database schema of a lender system (CRDB, NMB and MoFinance share the same structure) and the separate NIDA registry database. The tables are grouped by responsibility: authentication, customer profile, banking, lending, consent and integration, configuration, and the NIDA registry. The data dictionary in Section 4.1.1 describes every table.

DAIRE Lender Database Schema (CRDB / NMB / MoFinance share this structure)

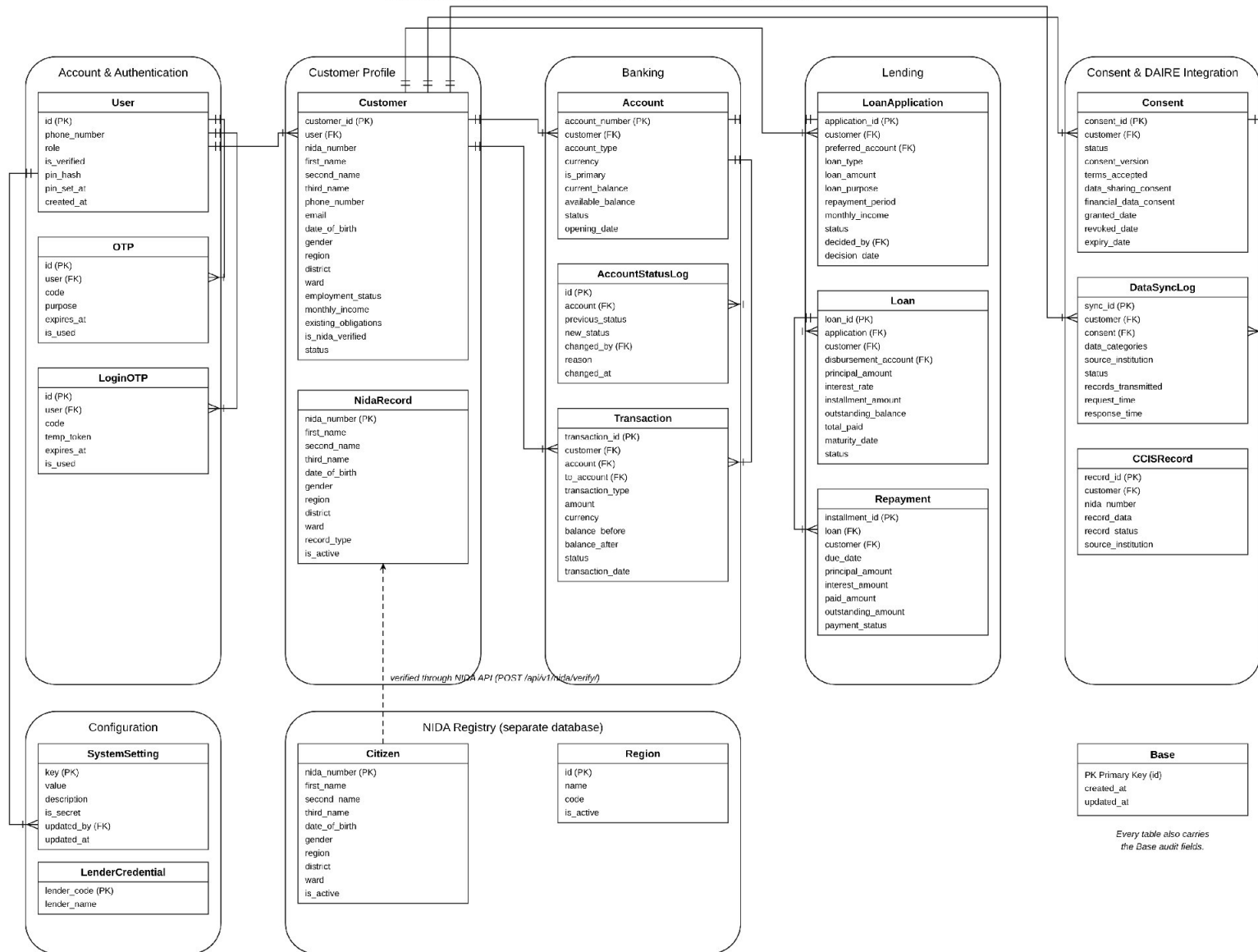


Figure 4: Entity-relationship diagram

4.1.1 Data Dictionary

The tables below list the main fields of each entity. Every table also has an automatically generated primary key and audit timestamps where shown.

Table: User

Login identity of every customer and staff member

Field	Type	Description
id	SERIAL	Primary key.
phone_number	VARCHAR	Phone number used as the login identity.
role	VARCHAR	Role of the user (for example customer or administrator).
is_verified	BOOLEAN	True after the account has been verified.
pin_hash	VARCHAR	Salted hash of the four-digit PIN (never the PIN itself).
pin_set_at	TIMESTAMP	When the PIN was last set.
created_at	TIMESTAMP	Creation time.

Table: OTP

One-time passwords for registration and password reset.

Field	Type	Description
id	SERIAL	Primary key.
user	INTEGER (FK)	Owner of the code (User).
code	VARCHAR	The one-time code.
purpose	VARCHAR	Why the code was issued.
expires_at	TIMESTAMP	Expiry time (ten minutes after creation).
is_used	BOOLEAN	True once consumed.

Table: Customer

Customer profile held by the lender

Field	Type	Description
customer_id	VARCHAR	Primary business identifier (also the borrower reference used with Central).
user	INTEGER (FK, unique)	Linked login identity (User).

Field	Type	Description
nida_number	VARCHAR	National identification number (20 digits).
first_name	VARCHAR	First name.
second_name	VARCHAR	Second name.
third_name	VARCHAR	Third name (optional).
phone_number	VARCHAR	Phone number.
date_of_birth	DATE	Date of birth from NIDA.
gender	VARCHAR	Gender from NIDA.
employment_status	VARCHAR	Employment status.
monthly_income	DECIMAL	Declared monthly income (TZS).
is_nida_verified	BOOLEAN	True when NIDA confirmed the identity.
status	VARCHAR	Customer status.

Table: NidaRecord

Local copy of the verified NIDA citizen data kept by the lender.

Field	Type	Description
nida_number	VARCHAR	National identification number.
first_name	VARCHAR	First name.
second_name	VARCHAR	Second name.
third_name	VARCHAR	Third name.
date_of_birth	DATE	Date of birth.
gender	VARCHAR	Gender.
record_type	VARCHAR	Type of record (for example NIDA).
is_active	BOOLEAN	Active flag.

Table: Account

Bank account owned by a customer.

Field	Type	Description
account_number	VARCHAR	Unique account number.
customer	INTEGER (FK)	Owner (Customer).
account_type	VARCHAR	Type, for example savings.

Field	Type	Description
currency	VARCHAR	Currency (TZS).
is_primary	BOOLEAN	True for the primary account.
current_balance	DECIMAL	Ledger balance.
available_balance	DECIMAL	Balance available to spend.
status	VARCHAR	Account status (active, frozen, closed).

Table: Transaction

Every deposit, withdrawal and transfer.

Field	Type	Description
transaction_id	VARCHAR	Unique transaction identifier.
customer	INTEGER (FK)	Customer.
account	INTEGER (FK)	Account debited or credited.
to_account	INTEGER (FK)	Destination account for transfers.
amount	DECIMAL	Amount in TZS.
balance_before	DECIMAL	Balance before the transaction.
balance_after	DECIMAL	Balance after the transaction.
status	VARCHAR	Transaction status.

Table: LoanApplication

Loan request submitted by a customer.

Field	Type	Description
application_id	VARCHAR	Unique application reference.
customer	INTEGER (FK)	Applicant.
preferred_account	INTEGER (FK)	Account to receive the loan.
loan_type	VARCHAR	Type of loan.
loan_amount	DECIMAL	Requested amount (TZS).
loan_purpose	VARCHAR	Purpose.

Field	Type	Description
repayment_period	INTEGER	Repayment period in months.
monthly_income	DECIMAL	Income declared in the application.
status	VARCHAR	Pending, approved or rejected.

Table: Loan

Approved and disbursed loan. (Django app: loans).

Field	Type	Description
loan_id	VARCHAR	Unique loan identifier.
application	INTEGER (FK, unique)	Application it came from.
customer	INTEGER (FK)	Borrower.
disbursement_account	INTEGER (FK)	Account that received the funds.
interest_rate	DECIMAL	Interest rate.
installment_amount	DECIMAL	Amount of each installment.
outstanding_balance	DECIMAL	Balance still owed.
total_paid	DECIMAL	Total repaid so far.
status	VARCHAR	Active, completed or defaulted.

Table: Repayment

Installment schedule and payment status of a loan. (Django app: repayments).

Field	Type	Description
installment_id	VARCHAR	Unique installment identifier.
loan	INTEGER (FK)	Loan.
customer	INTEGER (FK)	Borrower.
due_date	DATE	Due date.
principal_amount	DECIMAL	Principal part.
interest_amount	DECIMAL	Interest part.
paid_amount	DECIMAL	Amount paid.

Field	Type	Description
outstanding_amount	DECIMAL	Amount still due.
payment_status	VARCHAR	On time, late, missed or pending.

Table: LenderCredential

Registry of lenders and their callback addresses. (Django app: system_settings).

Field	Type	Description
lender_code	VARCHAR	Lender code, for example CRDB-01.
lender_name	VARCHAR	Lender name.
callback_url	VARCHAR	Address where Central broadcasts results.
lender_id	VARCHAR	Lender identifier in Central.
is_active	BOOLEAN	Active flag.
registered_at	TIMESTAMP	Registration time.

Table: Citizen

National identity record held by the NIDA system. (Django app: NIDA registry).

Field	Type	Description
nida_number	VARCHAR	National identification number (20 digits, unique).
first_name	VARCHAR	First name.
second_name	VARCHAR	Second name.
third_name	VARCHAR	Third name.
date_of_birth	DATE	Date of birth.
gender	VARCHAR	Gender.
is_active	BOOLEAN	True for active citizens (only these are verified).
created_at	TIMESTAMP	Creation time.
updated_at	TIMESTAMP	Last update.

4.2 User Case Diagram

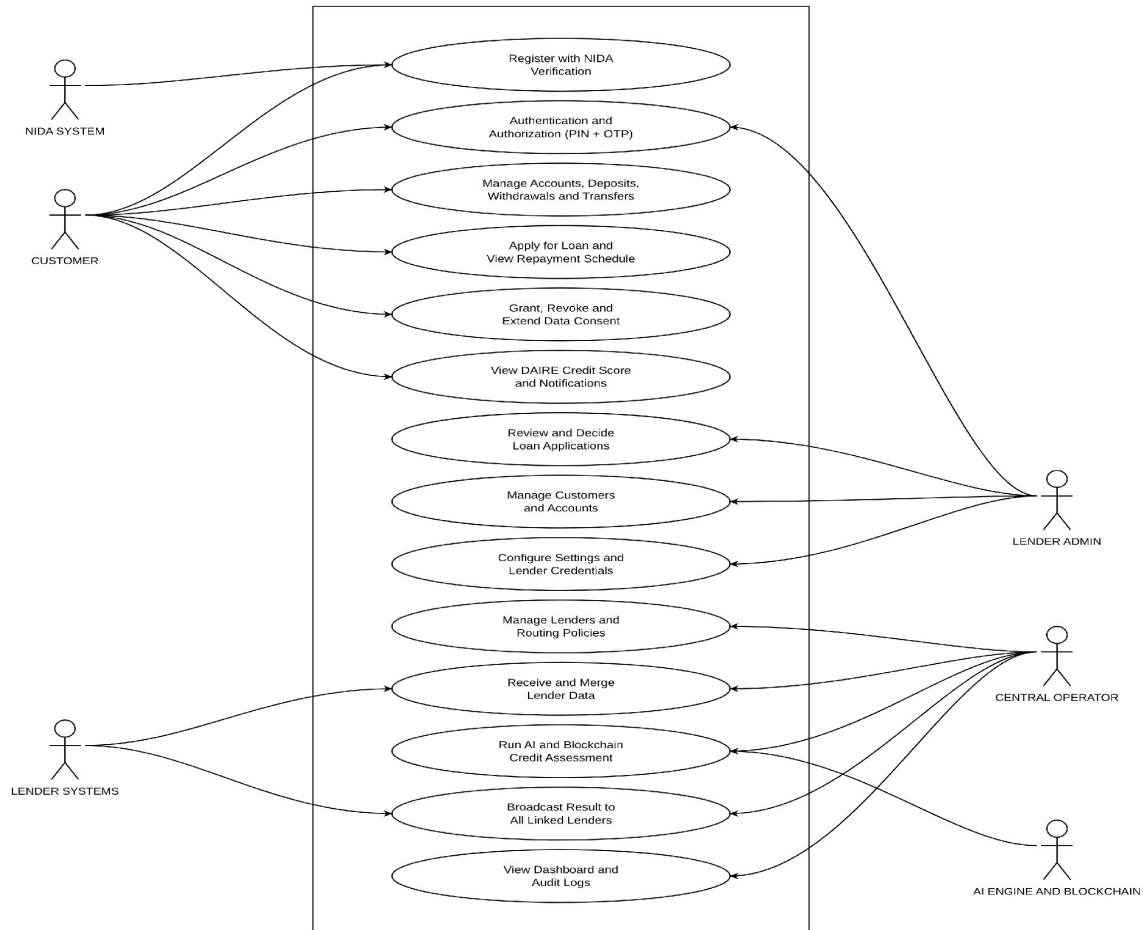


Figure 5: User Case Diagram

4.2.1 Actors

Actor	Description
Customer	A person who registers at a lender, manages accounts, applies for loans, controls consent and views the DAIRE credit score.
Lender Admin	Lender staff (loan officer or administrator) who review loan applications, manage customers and accounts and configure settings and lender credentials.
Central Operator	Staff of the Central System who manage lenders and routing policies, run assessments, broadcast results and monitor dashboards and audit logs.
NIDA System	External registry that verifies identity during registration.
Lender Systems	Lender backends that push consented data to Central and receive broadcasts.
AI Engine and	External scoring services called by Central to produce the reputation

Actor	Description
Blockchain	and score.

4.2.2 Use Case Descriptions

ID	Use Case	Actors	Description
UC-001	Register with NIDA Verification	Customer, NIDA System	The customer enters the NIDA number and names; the lender verifies them with NIDA, then creates the customer and primary account.
UC-002	Authentication and Authorization	Customer, Lender Admin	The user logs in with account number and PIN and confirms with a one-time password; a JWT access token and a refresh token are issued.
UC-003	Manage Accounts, Deposits, Withdrawals and Transfers	Customer	The customer views balances and transactions and moves money between accounts.
UC-004	Apply for Loan and View Repayment Schedule	Customer	The customer submits a loan application and, after approval, sees the installment schedule.
UC-005	Grant, Revoke and Extend Data Consent	Customer	The customer controls whether the lender may share data with the Central System.
UC-006	View DAIRE Credit Score and Notifications	Customer	The customer sees the score, risk band, credit history and notices received from Central.
UC-007	Review and Decide Loan Applications	Lender Admin	The officer reviews an application (optionally with the central score) and approves or rejects it.
UC-008	Manage Customers and Accounts	Lender Admin	The administrator views customers and changes account status.
UC-009	Configure Settings and Lender Credentials	Lender Admin	The administrator maintains runtime settings and the lender registry used for the Central connection.
UC-010	Manage Lenders and Routing Policies	Central Operator	The operator registers lenders and defines which fields go to the AI engine and to the blockchain.
UC-011	Receive and Merge Lender Data	Central Operator, Lender Systems	Central accepts lender pushes and merges them on the borrower reference.
UC-012	Run AI and	Central Operator,	Central creates an assessment, scores

ID	Use Case	Actors	Description
	Blockchain Credit Assessment	AI Engine and Blockchain	it with AI and the smart contract and stores the results.
UC-013	Broadcast Result to All Linked Lenders	Central Operator, Lender Systems	Central posts the identical result to each lender linked to the borrower.
UC-014	View Dashboard and Audit Logs	Central Operator	The operator monitors counts, data exchanges, deliveries and administrator actions.

5. SYSTEM INTERFACE DESIGN

5.1 User Interface Design

The user interface design models the functionality of the DAIRE web applications. It focuses on providing an intuitive, user-friendly experience on computers and tablets, ensuring accessibility, responsiveness and ease of navigation. Key design principles include minimalism to reduce cognitive load, clear feedback for actions such as loan submission and consent changes, and secure input fields for sensitive data such as PINs and one-time passwords. The four lender systems use the same screens with different branding and layout, so a customer experiences the same functions at every lender.

The interfaces are built with Angular and a component-based architecture. They are grouped into three applications.

5.1.1 Customer Portal (Lender)

1. Landing and authentication screens: login with account number and PIN, one-time password entry, registration with NIDA verification and forgot-password.
2. Customer dashboard (for example the CRDB dashboard at port 4600) showing balances and recent activity.
3. Accounts and transactions screens for deposits, withdrawals and transfers.
4. Loans screens with the application form, application status and repayment schedule.
5. Consent screen to grant, revoke and extend data sharing with the Central System.
6. Credit history and DAIRE score screen showing the score, risk band and notices from Central.
7. Savings groups, notifications and profile screens.

5.1.2 Lender Administration Portal

1. Administrator login and dashboard (for example the NMB loan applications page at port 4300).
2. Loan applications list with approve and reject actions and the central score of the applicant.
3. Customers and accounts management, including account status changes.
4. System settings and lender credentials used for the connection to the Central System.
5. Audit log viewer.

5.1.3 Central Operations Console

1. Dashboard with counts of lenders, borrowers, assessments and exchanges.
2. Lenders and routing policies management.
3. Borrowers view with accounts, loans, repayments and data conflicts.
4. Data Exchange page (port 4200) showing every push, pull and broadcast with its status.
5. Assessments page to run the AI reputation and the blockchain score and to broadcast the result.
6. Audit logs of administrator actions.

5.2 External Interfaces

The system interacts with external systems and between its own systems through well-defined APIs. These interfaces ensure seamless integration while maintaining security and performance.

5.2.1 NIDA Verification API

Item	Description
Endpoint	POST /api/v1/nida/verify/ (health check: GET /api/v1/nida/health/)
Access	Callers must be in the TRUSTED_CLIENT_IPS list; a read-only database role is used for lookups.
Request	nida_number (20 digits), first_name, second_name, third_name (optional)
Success response	matched = true, reason, citizen (name, date of birth, gender, region, district, ward)

5.2.2 Central Lender Data and Broadcast APIs

Item	Lender to Central	Central to Lender
Endpoint	POST /api/lender-data/receive/	POST {lender base}/api/daire/central/receive/
Key field	borrower_reference (top level and inside payload must be equal)	borrower_reference as held by the lender; central_borrower_reference

Item	Lender to Central	Central to Lender
		for audit
Content	Customer profile, accounts, loans (with repayments), transactions, consent evidence	assessment_reference and both results (AI and blockchain) in one wrapper
Protection	Active consent; optional API key when REQUIRE_API_KEYS is enabled	Lender callback IP must be in TRUSTED_CALLBACK_IPS
Failure	HTTP 400 for mismatching references or invalid payload	HTTP 404 when the lender does not hold the borrower; delivery marked failed

5.2.3 Lender Backend API Catalogue

The lender backend exposes the following main REST endpoints (all under /api/):

Area	Endpoint	Method	Purpose
Auth	/api/auth/verify-nida/	POST	Verify NIDA number and names before registration
Customer	/api/customers/me/	GET, PATCH	View and update the logged-in customer
Accounts	/api/accounts/	GET	List customer accounts
Transactions	/api/transactions/deposit/, /withdraw/, /transfer/	POST	Money movements
Loans	/api/loans/applications/	GET, POST	List and submit loan applications
Loans	/api/loans/applications/{id}/approve/ and /reject/	POST	Officer decision
Loans	/api/loans/applications/{id}/central-score/	GET	Central score for an applicant
Repayments	/api/repayments/{id}/mark-paid/	POST	Record a payment of an installment
Consent	/api/consent/{id}/grant/, /revoke/ and extend	POST	Consent management
Savings	/api/savings/ and /api/savings/records/	GET, POST	Savings and contributions
Credit	/api/credit-history/ and /api/financial-behaviour/	GET	Customer credit history and behaviour

Area	Endpoint	Method	Purpose
DAIRE	/api/daire/central/receive/	POST	Receive broadcast from Central

5.2.4 Central System API Catalogue

Endpoint	Method	Purpose
/api/lenders/	GET, POST, PATCH	Register and maintain lenders and callback addresses
/api/borrowers/	GET	Unified borrower records
/api/borrowers/pull-from-all-lenders/	POST	Pull data for a borrower from every lender
/api/borrowers/{id}/broadcast-result/	POST	Broadcast the result to all linked lenders
/api/lender-data/receive/	POST	Lender push of borrower data
/api/assessments/	GET, POST	Create and list assessments
/api/assessments/{reference}/ai-reputation/	POST	Run the AI reputation
/api/assessments/{reference}/blockchain-score/	POST	Run the blockchain score
/api/routing-policies/	GET, POST	Field routing to AI and blockchain
/api/data-exchanges/	GET	Log of every exchange with external systems
/api/dashboard/	GET	Dashboard counts
/api/audit-logs/	GET	Administrator audit trail
/api/v1/score/submit (middleware)	POST	Submit features to the contract
/api/v1/score/{borrowerRef} (middleware)	GET	Read the stored score

6.SYSTEM COMPONENTS

6.1 Frontend

Technology: Angular (TypeScript) for dynamic and responsive web interfaces, with a separate application for each lender and for the Central operations console.

Key Features:

1. User authentication with PIN and one-time password, role-based dashboards for customers and administrators.
2. Loan application, account and consent management, with form validation and clear error messages.
3. Display of the DAIRE credit score, risk band and credit history received from Central.
4. Central console pages for lenders, borrowers, assessments, routing policies and data exchange.
5. Branding per lender (CRDB, NMB, MoFinance) using the same functional components.

6.2 Backend

Technology: Django (Python) with Django REST Framework and JWT authentication for the lender and NIDA systems, and a Node.js Express middleware for the blockchain.

Key Features:

1. REST API for the front ends, with OpenAPI documentation.
2. Business logic for customers, accounts, loans, repayments, consent and synchronization, including validation rules and atomic money movements.
3. NIDA client for identity verification and a NIDA service with an IP allow-list.
4. Central services for borrower merge, feature generation, AI reputation, blockchain score and broadcast.

6.3 Database

Technology: PostgreSQL for structured data storage, chosen for its robustness and ACID compliance.

6.4 AI Reputation Engine

The AI engine converts the borrower's credit features into a reputation label and a default probability. Central uses a trained scikit-learn Random Forest model loaded from a joblib file unless an external engine is configured with AI_ENGINE_URL. If the model cannot be loaded, a fallback formula is used so that scoring does not stop.

Feature	Meaning
active_loan_count	Number of loans that are still active
completed_loan_count	Number of loans fully repaid
defaulted_loan_count	Number of loans in default
total_outstanding_debt	Total unpaid balance
on_time_payment_ratio	Share of payments made on time (0 to 1)
missed_payment_count	Payments never made
late_payment_count	Payments made late
max_days_overdue	Longest delay on any repayment, in days

Feature	Meaning
transaction_frequency	Number of transactions in the reporting period
income_frequency	Number of income transactions
balance_stability	Stability of the account balance

The response contains the reputation (EXCELLENT, GOOD, MODERATE or HIGH_RISK), the score, an optional risk level, a behaviour summary and the model version. A response that lacks reputation, score or model_version is rejected and the caller receives HTTP 503.

6.5 Blockchain Scoring Service

The smart contract DaireCreditScore (Solidity) calculates the credit score on the blockchain and stores it on the ledger. Five dimensions are scored from 0 to 100 and combined with fixed weights into a score between 300 and 850:

Dimension	Evidence Used	Weight
D1 Payment Reliability	On-time ratio, maximum days late, missed count, streak	30%
D2 Financial Stability	Active months, regular months, balance stability	20%
D3 Credit and Debt Management	Defaults, completed loans, utilization, number of lenders	15%
D4 Behavioural Consistency	Trend, volatility, recency of a bad event	20%
D5 Verified Trust Evidence	History, verification, breadth of sources, conflicts	15%

The weighted sum (maximum 10,000) is converted by $S = 300 + (\text{weighted sum} \times 550) / 10,000$. The risk band is then assigned from the score:

Score	Risk Band
Below 580	POOR
580 to 669	FAIR
670 to 739	GOOD
740 to 799	VERY_GOOD
800 and above	EXCEPTIONAL

A dimension below 40 prevents the band from being better than FAIR, even if the total score is high. The contract exposes read functions (getScore, getDimensions, getAssessment, historyLength) and the pure preview function previewScore that calculates a result without storing it. The Express middleware (scoreEngine.js and index.js) connects Central to the contract through POST /api/v1/score/submit and GET /api/v1/score/{borrowerRef}.

6.6 NIDA Registry System

The NIDA system is an independent Django application with its own PostgreSQL database. It keeps the citizen records (Region and Citizen tables) and answers verification requests from trusted lenders. It runs at 172.17.16.70 on port 8020. Requests are accepted only from IP addresses in TRUSTED_CLIENT_IPS, and external systems may read citizens through the read-only role nida_api_reader and the view api_citizen. The registry was loaded from a PostgreSQL dump containing 200 citizens.

6.7 Central System Modules

Module	Responsibility
Lenders	Registry of lenders (CRDB-01, MoFinance-004, NMB-01, NMB-02) with callback addresses
Borrowers	Unified borrower with accounts, loans, repayments and recorded data conflicts
Assessments	Credit profile, features and results of each assessment
AI Reputation and Smart Contract Results	Stored outputs of the AI engine and the blockchain
Data Exchanges	Log of every push, pull, AI call, blockchain call and broadcast

7. SECURITY DESIGN

7.1 Authentication and Authorization

The system employs robust mechanisms to verify user identity and control access, preventing unauthorized actions and ensuring data protection.

- ❖ **PIN and one-time password:** A four-digit PIN is stored only as a hash. A one-time password valid for ten minutes is required for registration, login and password reset; a new login invalidates earlier unused codes.
- ❖ **JWT:** Django REST Framework with simplejwt issues an access token (12 hours) and a refresh token (7 days) after login.
- ❖ **Roles:** Permission classes restrict endpoints by role so that a customer can reach only their own data and only administrators can approve loans or change settings.

7.2 Network and Interface Security

1. NIDA protection: Only IP addresses listed in TRUSTED_CLIENT_IPS can call the verification endpoint.
2. Callback protection: Central may call a lender only when the lender address is in TRUSTED_CALLBACK_IPS.

3. API keys: Optional API keys can be required on the lender endpoints trusted-network deployments run keyless by default.
4. Secrets: API keys, database passwords and signing keys are kept in environment files and never in source code or logs.

7.3 Consent Management

Consent is the gate for all data sharing. A lender can push data only while a consent is active. Each consent stores its version, accepted terms, granted and revoked dates, expiry date and the IP address of the customer. When a customer revokes the consent, the lender stops all further pushes and records an audit entry. Consent can be extended with a new expiry date.

7.4 Data Privacy and Compliance

The system adheres to privacy best practices to protect user data and meet regulatory requirements.

1. Data Minimization: Only consented categories are sent to Central. The AI engine and the blockchain receive derived features and dimensions only, never the NIDA number, names, phone number or address.
2. Compliance: The system is designed to comply with the Tanzania Personal Data Protection Act, 2022, including consent management for data collection and the right of the customer to withdraw consent.
3. Auditability: Every data exchange, consent change and assessment is logged with a reference and a time.
4. Retention and Deletion: Personal data stays in the lender and NIDA databases; the ledger holds only the score, dimensions, band and timestamp keyed by borrower reference.

8. APPENDICES

8.1 Glossary of Terms

Term	Definition
DAIRE	Decentralized AI Reputation Engine, the platform described in this document
FSP	Financial Service Provider, a bank or financial institution that gives credit
NIDA	National Identification Authority, which keeps the national identity registry
Central System	The DAIRE system that merges lender data and coordinates scoring
Borrower Reference	The single identifier shared by all lenders for the same person
Assessment Reference	The identifier of one credit assessment run by Central
Consent	The customer's permission for the lender to share data with Central
API	Application Programming Interface, a set of tools and protocols for

Term	Definition
	software to talk to software
REST	An API style based on HTTP requests and JSON responses
Django	A high-level Python web framework that encourages rapid development and clean design
DRF	Django REST Framework, an extension for building web APIs
Angular	A TypeScript framework for building web interfaces
PostgreSQL	An open-source relational database management system
JWT	JSON Web Token, a compact means of representing claims transferred between two parties
OTP	One-Time Password, a code that can be used once and expires
PIN	Personal Identification Number, here a four-digit secret stored as a hash
Random Forest	A machine-learning algorithm that combines many decision trees
Smart Contract	A program on the blockchain that runs exactly as written
Solidity	The programming language used to write the smart contract
Hardhat	A development blockchain node used to run and test the contract
Risk Band	Class of the score: POOR, FAIR, GOOD, VERY_GOOD or EXCEPTIONAL
ACID	Atomicity, Consistency, Isolation, Durability, properties of reliable database transactions
RBAC	Role-Based Access Control, regulating access according to the role of the user
HTTPS/TLS	Protocols for encrypted links between networked computers
TZS	Tanzanian shilling, the currency used in the system

8.2 Compliance and Licensing

8.2.1. Software Licensing

Component	Licence
Django	BSD License (open-source)
Django REST Framework	BSD License (open-source)
Django REST Framework simplejwt	MIT License (open-source)
Angular	MIT License (open-source)

Component	Licence
PostgreSQL	PostgreSQL License (open-source)
scikit-learn	BSD License (open-source)
Node.js Express	MIT License (open-source)
Hardhat	MIT License (open-source)
Solidity	GPL-3.0 License (open-source)

8.3 Data Privacy and Compliance

Local Data Protection Law: All personal and sensitive information handled by the system will comply with Tanzanian government regulations such as the Personal Data Protection Act, 2022, and with the e-Government Authority standards.